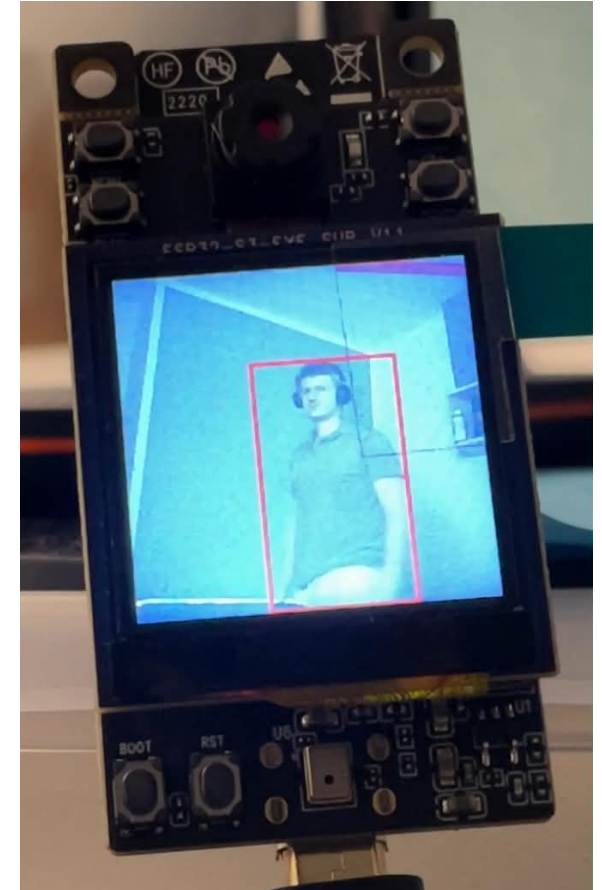


Smart Visual Alarm System

Erneyi Zoltán | MAT. 265056

Frames stay on the board. Only alarm metadata leaves the device.



Requirements

Embedded vision

- ESP32-S3-EYE camera frame acquisition
- LCD preview with detection overlay
- camera frames stay on the device

On-device AI

- ESP-WHO / ESP-DL pedestrian detector
- confidence threshold for valid candidates
- bounding-box output for visual feedback

Event control

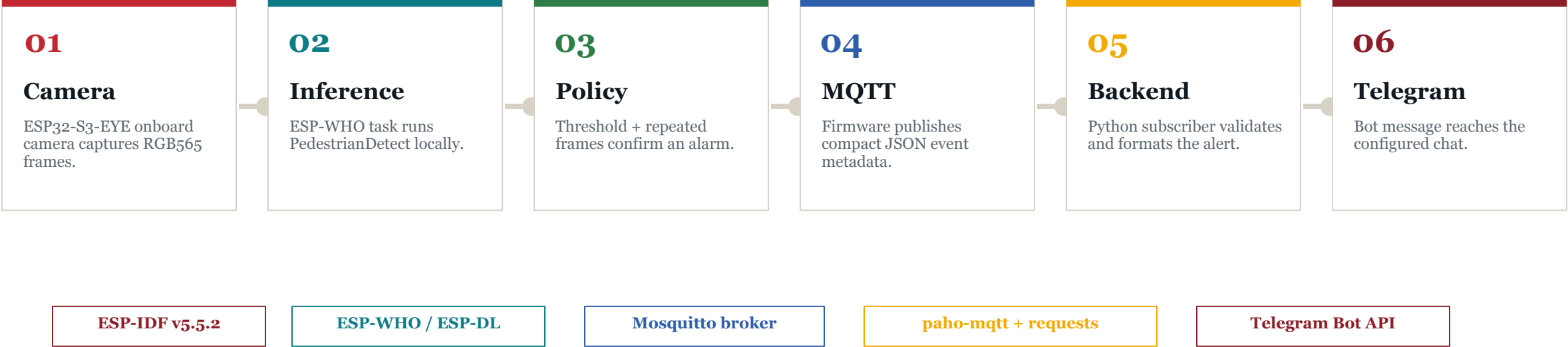
- confirmation frames before an alarm
- cooldown to avoid repeated alerts
- compact JSON metadata payload

Notification bridge

- MQTT topic transport
- Python validation and formatting
- Telegram Bot API user notification

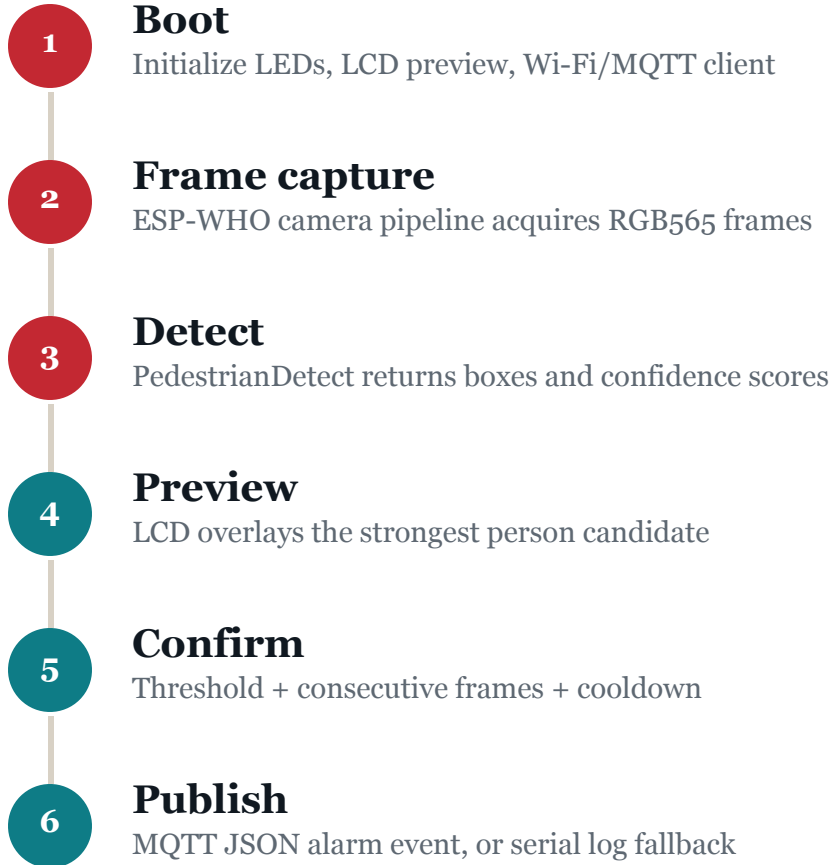
Core flow: detect locally -> confirm event -> publish MQTT -> notify Telegram

Six-stage alarm path



Boundary rule: raw camera frames stay on the ESP32-S3-EYE. The network event contains label, confidence, count, time, and box coordinates.

ESP32-S3-EYE runtime loop



Tunable Kconfig controls

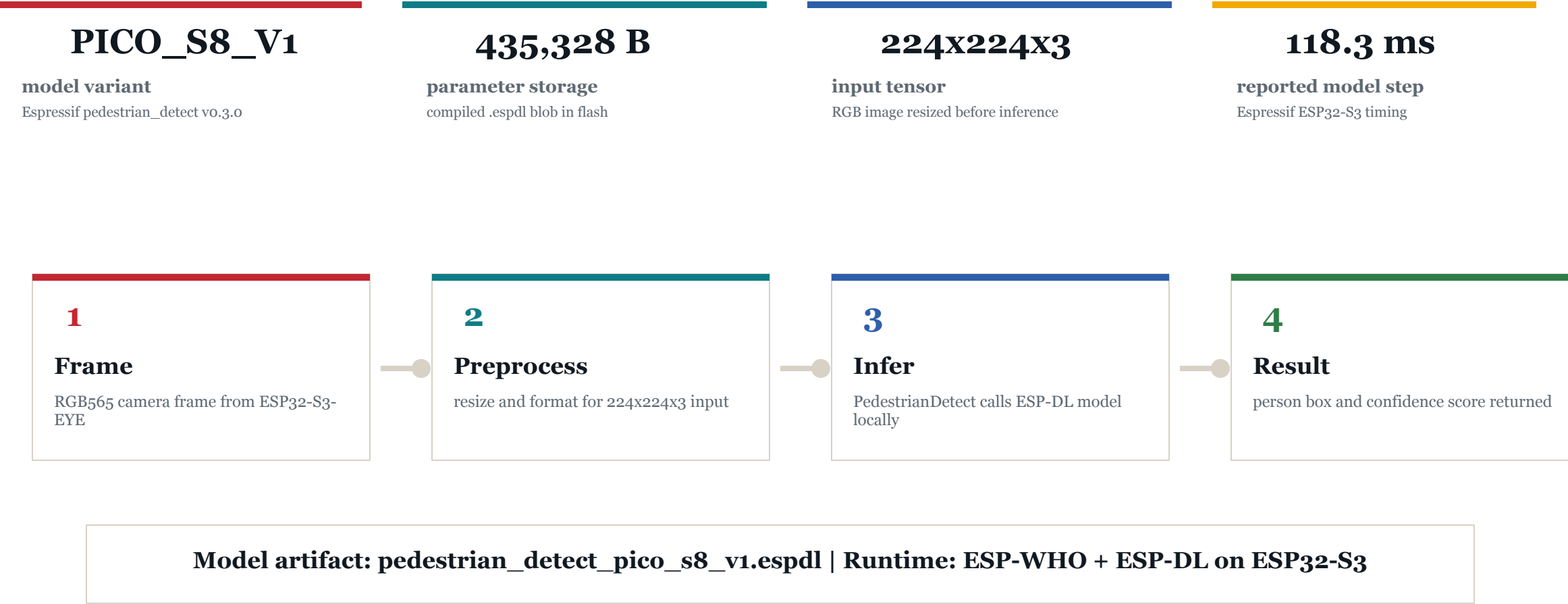
- score threshold
- confirmation frame count
- alert cooldown seconds
- device ID, broker URI, topic

Defensive networking

If Wi-Fi or MQTT is not configured, detections still run and alarm JSON is logged over serial.

Core files: `app_main.cpp`, `alarm_mqtt.cpp`,
`lcd_preview.cpp`, `frame_cap_pipeline.cpp`

The AI model runs fully on the ESP32-S3



MQTT events



```
{  
  "device_id": "esp32s3-eye-01",  
  "label": "person",  
  "confidence": 0.824,  
  "confirmation_frames": 11,  
  "box": {"x_min": 0, "y_min": 58}  
}
```

6+

required fields

event, device, time, label, confidence, frames

0..1

alarm quality

confidence score kept in payload

box

debug evidence

x_min, y_min, x_max, y_max

Design reason: the backend receives enough context for an alert, while the camera frame stays local.

Python bridge sends Telegram alerts



Backend checks

- required fields exist and have correct types
- confidence is numeric and between 0 and 1
- cooldown prevents repeated Telegram spam

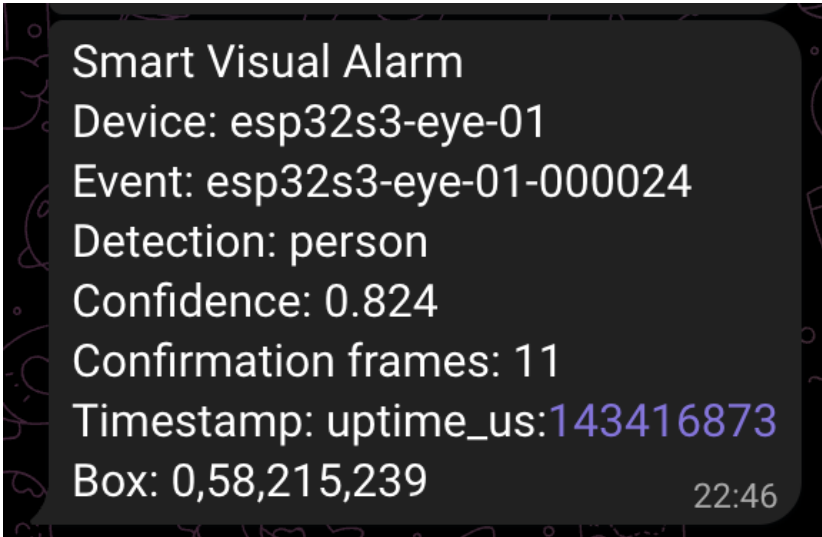
User sees: device, event ID, person label, confidence, confirmation frames, timestamp, and box.

The bounding box is visible on the device



ESP32-S3-EYE LCD preview with red person bounding box

Telegram alert metadata

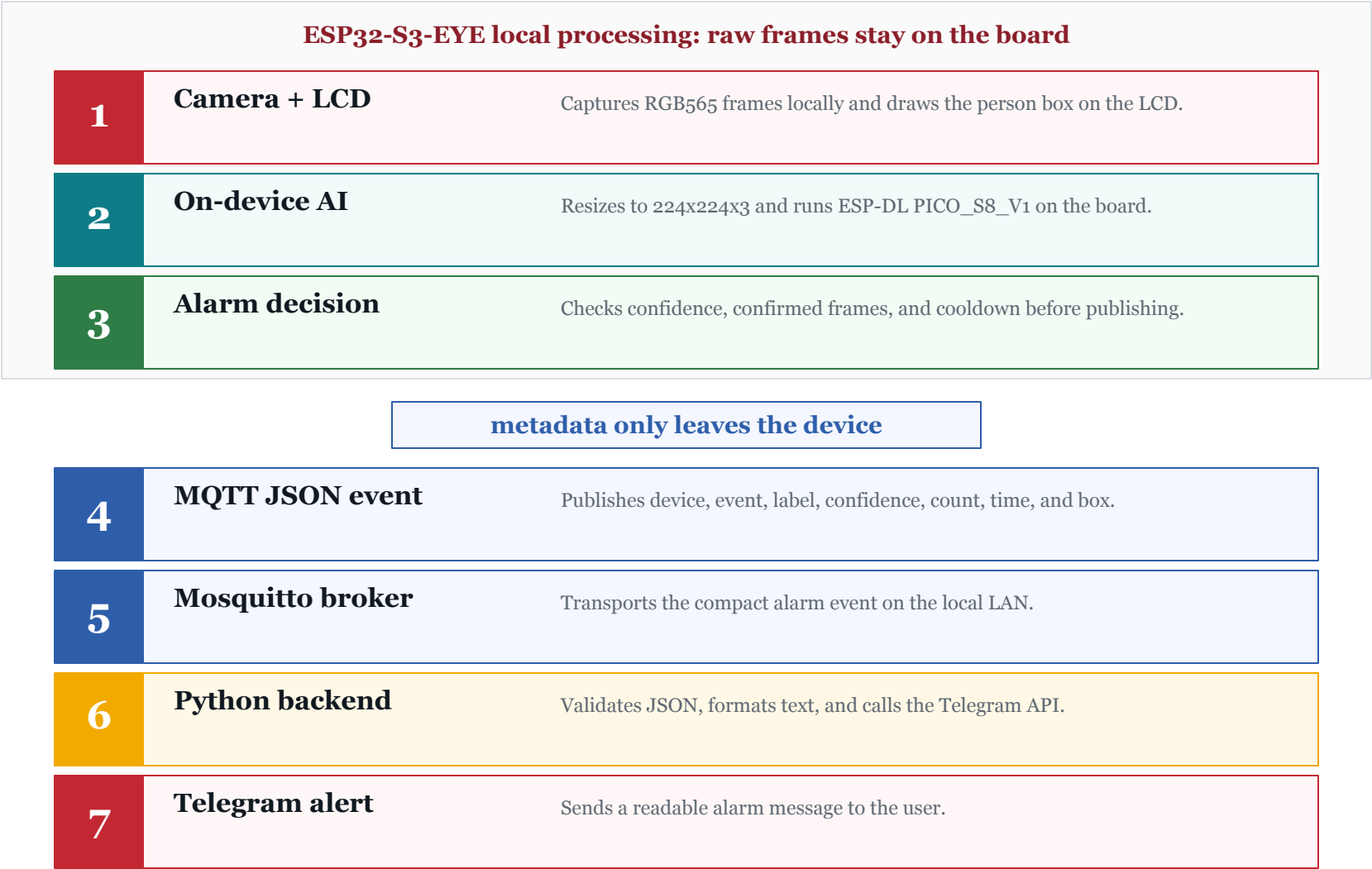


Telegram alert generated by the Python backend

1	Device	which ESP32-S3-EYE produced the alarm
2	Event	unique event ID for logs and debugging
3	Detection	detected class label, here: person
4	Confidence	model score used to understand alert quality
5	Confirmation frames	how many positive frames supported the alert
6	Timestamp	device uptime when the alert was created
7	Box	bounding-box coordinates from the detector output

The backend forwards readable metadata, not raw camera frames.

End-to-end system flow



Possible next steps

Reliability testing

- test empty scenes, motion, low light, backlight
- record false positives and missed detections
- compare thresholds and confirmation-frame settings

Latency metrics

- measure camera-to-MQTT timing
- measure MQTT-to-Telegram timing
- separate model latency from network latency

Deployment hardening

- broker authentication or TLS
- watchdog and reconnect handling
- token rotation and cleaner secret handling

Product features

- web dashboard for event history
- configurable zones or schedules
- optional image snapshot with privacy controls

Summary

Implemented capabilities

- ESP32-S3-EYE camera, LCD, and detector pipeline
- on-device pedestrian model with confidence output
- confirmation and cooldown alarm policy
- MQTT JSON event publishing
- Python Telegram notification bridge